AMEM-Net: Adaptive Multi-Expert Memory Network for Self-Supervised Industrial Visual Anomaly Detection

CHIGURUPATI VENKAT SAI KIRAN[1], DR. REENA ROY[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamil Nadu 600127, India (e-mail: chigurupati.venkat2025@vitstudent.ac.in; reenaroy.r@vit.ac.in).

Corresponding author: Chigurupati Venkat Sai Kiran (e-mail: chigurupati.venkat2025@vitstudent.ac.in).

ABSTRACT Industrial visual inspection often has many defect-free examples but few labelled examples of defective products. This makes it useful to learn normal appearance and identify deviations without requiring a complete catalogue of defects. This paper presents AMEM-Net, an adaptive multi-expert feature-memory method for industrial visual anomaly detection. The implementation uses frozen ImageNet-pretrained WideResNet50-2 features from two intermediate stages, clusters normal image descriptors into four groups, and builds a patch memory for each group. A trainable router assigns input-dependent expert weights, and memory-distance feedback adjusts the expert contributions before patch scores are aggregated into an image-level anomaly score. The router is trained with normal images and synthetic local perturbations; the decision threshold is calibrated on validation data. The evaluation covers eight MVTec AD categories and compares CNN-AE, ResNet-AE, MemAE, PatchCore, and AMEM-Net using AUROC, F1-score, precision, recall, and accuracy. Across the eight categories, AMEM-Net achieves macro-mean AUROC of 97.72% and F1-score of 94.84%, compared with 97.00% and 92.33%, respectively, for PatchCore. These results show the performance of the implemented method under the documented evaluation protocol. The paper describes the architecture, evaluation settings, complete category-wise results, and directions for extending the study to additional categories and localization analyses.

INDEX TERMS Anomaly detection, computer vision, industrial inspection, memory networks, self-supervised learning, visual anomaly localization.

I. INTRODUCTION

Visual inspection is a routine part of manufacturing quality control, yet the appearance of a defect is difficult to enumerate in advance. A production line may provide thousands of acceptable products while only a small number of defective examples are available, and the next defect may differ from those seen before. Industrial anomaly detection addresses this imbalance by learning normal appearance from defect-free images and assigning larger anomaly scores to samples that depart from that learned representation. MVTec AD established a widely used benchmark for this setting by providing normal training images, mixed normal and anomalous test images, and pixel-level defect masks across object and texture categories [1], [2].

Published approaches use different signals to identify departures from normality. Reconstruction-based systems score image or feature reconstruction error; distribution-based methods estimate the likelihood of local features; memory-bank approaches compare test features with stored normal examples; and teacher–student methods measure feature-prediction discrepancies. These families have different strengths and failure modes, and no single design is uniformly preferable across every category or operating condition [3]–[8]. In particular, PatchCore demonstrates that a carefully selected memory of pretrained patch features can be a strong baseline, so an additional memory mechanism must be evaluated against it rather than assumed to improve performance.

This work studies one specific design question: whether normal patch descriptors can be divided into multiple expert memories and combined through input-dependent routing. A product image may contain several visual modes, such as smooth surfaces, repeated textures, edges, or small components. A single memory can represent such variation, but its retrieval behavior depends on which examples are retained and how their distances are combined. The working hypothesis is that expert memories, together with a router that responds

to each input, may provide a useful alternative to a single undifferentiated memory bank. The study evaluates this design against established reconstruction-based and memory-bank baselines.

We implement Adaptive Multi-Expert Memory Network (AMEM-Net). The feature extractor is an ImageNet-pretrained WideResNet50-2 whose weights remain frozen. Layer-2 descriptors are concatenated with upsampled Layer-3 descriptors to form 1,536-dimensional patch features. Global descriptors from normal training images are clustered into four groups; patch descriptors from each group populate an expert memory with 512 entries. A small multilayer perceptron predicts input-dependent expert weights. A memory-distance feedback term modifies the weights, and the resulting expert scores are combined with the minimum distance across experts. Router training uses normal examples and synthetic local perturbations with a margin-based objective. Synthetic proxy tasks are also used in contrastive representation learning and feature-space anomaly generation, although their objectives differ from the router loss used here [9]–[11]. Threshold calibration uses validation scores and synthetic pseudo-anomalies, and the threshold is fixed before the test labels are used for final metric calculation.

The evaluation covers eight MVTec AD categories: bottle, carpet, hazelnut, metal nut, wood, leather, cable, and transistor. Five image-level measures are reported for CNN-AE, ResNet-AE, MemAE, PatchCore, and AMEM-Net: AUROC, F1-score, precision, recall, and accuracy. The macro mean over the eight categories is 97.72% AUROC and 94.84% F1 for AMEM-Net, compared with 97.00% and 92.33% for PatchCore, giving an AUROC difference of 0.72 percentage points. The results are interpreted within the selected eight-category benchmark and the threshold-calibration procedures described in the experimental setup.

The paper makes four contributions. First, it describes an implementation that combines four normal feature memories, input-dependent routing, memory-distance feedback, and score fusion. Second, it formalizes the implemented feature extraction, memory construction, routing, and anomaly-scoring steps. Third, it reports category-wise results and macro means for five model implementations across eight MVTec AD categories. Fourth, it analyses category-level performance and discusses how adaptive memory can be studied further for industrial visual inspection. The remaining sections review related methods, formulate the research questions, describe the implementation and evaluation protocol, and discuss the results.

II. RELATED WORK

A. RECONSTRUCTION-BASED ANOMALY DETECTION

A common strategy for unsupervised anomaly detection is to learn how normal images should be represented or reconstructed. An encoder–decoder is optimized using normal examples, and reconstruction discrepancy is interpreted as evidence of abnormality. This approach is straightforward and can produce dense error maps, but the discrepancy does not always correspond to a defect: normal variation may reconstruct poorly, while an unseen pattern may be reconstructed adequately. MemAE addresses this limitation by introducing a memory module that retrieves prototypical items and constrains the representation used for reconstruction [12]. The MemAE baseline follows the project implementation described in the experimental setup.

DRAEM combines reconstruction with a discriminative embedding and trains with synthetic anomalies, learning a representation that separates simulated defects from normal regions [13]. CutPaste also uses synthetic cut-and-paste transformations, but it first learns a self-supervised representation and then fits a one-class detector [14]. Uninformed Students uses a teacher–student discrepancy and an uncertainty signal to detect anomalous regions [15]. These approaches show that synthetic proxy tasks and pretrained representations are established design choices; AMEM-Net must therefore be distinguished by its specific memory partitioning and fusion mechanism, not by the mere use of pseudo-anomalies. FastFlow models feature distributions with two-dimensional normalizing flows [16]. EfficientAD uses a student–teacher feature prediction mechanism and an auxiliary autoencoder to target both local and logical anomalies while emphasizing inference efficiency [17]. Student–teacher feature pyramid matching and reverse distillation also use discrepancies between learned normal representations to identify anomalous regions [7], [8]. These methods illustrate that reconstruction error is one of several possible signals and that synthetic perturbations can be useful training tools, but synthetic anomalies do not guarantee transfer to the full range of real defects.

B. FEATURE DISTRIBUTIONS AND MEMORY BANKS

Feature-based methods often exploit pretrained convolutional networks because their intermediate representations preserve useful texture and shape information. PaDiM models the distribution of patch-level features using location-dependent Gaussian statistics and scores deviations from those distributions [5]. PatchCore instead stores a representative subset of normal patch features and measures the distance from a test patch to its nearest memory item. Coreset selection reduces redundancy while preserving a useful approximation to the normal feature set [6]. PatchCore is a particularly relevant baseline because the current AMEM-Net also uses frozen pretrained features and nearest-memory distances. Conditional-flow methods such as CFLOW-AD model feature likelihoods, whereas SimpleNet learns a discriminator over normal and synthetic anomaly features [11], [18].

Student–teacher methods form another major family. Student–Teacher Feature Pyramid Matching (STFPM) compares multi-scale features from a pretrained teacher and a student trained on normal data; discrepancies across feature levels provide local anomaly evidence [7]. Reverse distillation changes the direction of feature reconstruction by training a student decoder to recover a teacher representation from a

compact one-class embedding [8]. These methods can produce localization maps without an explicit RGB reconstruction objective. Their trainable student components, however, differ from the current AMEM-Net setup, where the pretrained feature extractor is frozen and the router is the principal trainable component.

CFLOW-AD models feature likelihoods through conditional normalizing flows and was designed with inference efficiency in mind [18]. SimpleNet generates synthetic anomaly features and trains a discriminator to separate them from normal features [11]. These methods are particularly relevant to the use of synthetic perturbations in AMEM-Net, but the synthetic data enter the learning process differently: in SimpleNet, synthetic features train an anomaly discriminator, whereas the current AMEM-Net objective uses synthetic image perturbations to shape expert routing. Their published benchmark scores are discussed as literature context, while the quantitative comparison in this paper uses the five implementations evaluated in the project.

Vision–language methods address a different challenge: reducing dependence on normal examples from the target category. WinCLIP uses text prompts and window-level visual representations for zero-shot or few-shot detection [19], while AnomalyCLIP learns object-agnostic prompts intended to generalize normality and abnormality concepts across domains [20]. Such methods are relevant to the candidate gap of unseen-category generalization. The present AMEM-Net evaluation is category-specific; these methods are useful references for a future study designed around held-out product categories.

Both PatchCore and AMEM-Net use normal feature representations. AMEM-Net partitions normal patch descriptors into four groups according to image-level normal modes, creates one memory per group, predicts input-dependent weights, and includes the minimum distance across experts in the combined score. The method is thus an adaptive fusion of multiple normal memories. A component-wise comparison of single and multiple memories is a useful extension for isolating the effect of expert partitioning from the effects of routing and score fusion.

C. MULTI-CATEGORY AND GENERALIZABLE METHODS

UniAD studies a unified model for multiple anomaly-detection categories and addresses shortcut behavior in reconstruction-based architectures [21]. WinCLIP explores zero-shot and few-normal-shot anomaly classification and segmentation using vision–language representations [19]. AnomalyCLIP further studies object-agnostic prompts for zero-shot anomaly detection across domains [20]. These settings are related to generalization but differ from the category-specific memory evaluation used here. Testing several categories separately does not constitute an unseen-category experiment: a category is unseen only when its training examples are excluded from model fitting and its evaluation protocol is explicitly designed to measure transfer.

MVTec AD contains fifteen object and texture categories, and the dataset authors provide a defined training and test protocol with pixel-precise annotations [1], [2]. The related MVTec LOCO AD benchmark explicitly includes logical anomalies, such as invalid object arrangements, that are not reducible to local surface defects [22]. The current study uses eight selected MVTec AD categories, spanning both textures and objects. Logical-constraint anomalies represented in MVTec LOCO AD and transfer to entirely held-out categories are complementary evaluation settings. Additional industrial datasets can broaden the assessment of the method under different acquisition conditions [23].

D. ADDITIONAL BENCHMARKS, REPRESENTATIONS, AND EVALUATION PRACTICE

The broader anomaly-detection literature includes methods that learn a compact description of normality as well as methods that synthesize proxy anomalies. Deep Support Vector Data Description formulates one-class learning directly as a compact feature-space objective [24]. Earlier generative approaches include AnoGAN and GANomaly, which use reconstruction or latent-space discrepancies to detect samples outside the normal training distribution [25], [26]. SPADE uses deep feature correspondences between a test image and normal reference images to support local anomaly localization [27]. These works provide context for the choice of feature-space scores and demonstrate that anomaly detection can be approached without a single fixed reconstruction formulation.

Recent methods continue to broaden the design space. PNI incorporates position and neighborhood information, while FR-PatchCore modifies PatchCore to address generalization issues associated with positional variation [28], [29]. Dual-modeling decouple distillation uses a distillation-based formulation to separate normal and anomalous cues [30]. PromptAD uses prompt learning with normal-only examples in a few-shot setting, while multimodal approaches combine visual and geometric information rather than relying on RGB features alone [31]–[33]. A supervised alternative has also been studied for complex industrial images, highlighting the distinction between unsupervised normal-only evaluation and settings that can use labelled defects [34]. These methods define relevant adjacent directions for future comparisons under a matched data and evaluation protocol.

Dataset choice defines the setting being studied. MVTec LOCO AD includes logical anomalies, VisA expands the object categories and image count, and MVTec 3D-AD examines anomalies in three-dimensional data [22], [35], [36]. These benchmarks offer complementary directions for extending industrial anomaly-detection evaluation beyond the eight selected MVTec AD categories. Industrial anomaly-detection surveys also emphasize describing the task setting and deployment conditions clearly [3], [4].

E. POSITIONING OF AMEM-NET

The proposed design combines established ingredients: frozen ImageNet features, normal feature memories, clustering, an MLP router, synthetic perturbations, and threshold calibration.

Its potentially distinctive element is the exact coupling of image-level mode-based expert memories, input-dependent routing, error-derived feedback weights, and a conservative minimum-expert score. AMEM-Net is presented as an adaptive multi-expert memory design that couples these components within a single anomaly-scoring pipeline. The related-work analysis explains how this design differs from the closest memory-bank and reconstruction-based approaches.

F. COMPARISON WITH REPRESENTATIVE ANOMALY-DETECTION FAMILIES

Table 1 positions the evaluated design relative to representative methods. It is a conceptual comparison of method families, not a table of directly comparable benchmark scores. The methods differ in training regime, feature backbone, output score, and whether their primary goal is category-specific, multi-category, few-shot, or zero-shot detection. This distinction matters because the same dataset name does not guarantee an identical evaluation protocol.

The comparison clarifies the scope of the current contribution. AMEM-Net is closest to PatchCore in its use of frozen pretrained patch features and nearest-memory distances. It is related to MemAE through the use of multiple stored normal representations, but the current AMEM-Net does not reconstruct RGB images. It is related to DRAEM through the use of synthetic perturbations, although DRAEM trains a discriminative reconstruction embedding while the current implementation uses synthetic perturbations to train the router. These distinctions are stated explicitly to avoid conflating the methods.

Recent methods such as vision–language detectors, multi-category models, and multimodal systems provide useful context for future comparative studies. Published results can differ in input resolution, category coverage, preprocessing, test-time aggregation, and threshold rules. The related-work discussion therefore compares their technical approaches and positions AMEM-Net within the broader research landscape.

III. PROBLEM FORMULATION AND RESEARCH QUESTIONS

Let an input image be $x \in \mathbb{R}^{3\times H_0\times W_0}$. Training data for a category consist only of normal images, $\mathcal{D}_N = \{x_i\}_{i=1}^{N}$. Test images may be normal or anomalous, and pixel masks are available for the anomalous samples in MVTec AD. The objective is to estimate an image anomaly score $s(x)$ and a patch-level anomaly map $A(x)$ such that larger scores indicate greater deviation from normal appearance.

This project investigates three questions. *RQ1:* How does a four-expert memory with adaptive fusion perform against reconstruction-based baselines and PatchCore on eight selected categories? *RQ2:* How does performance vary across categories? *RQ3:* How do threshold-calibration choices relate to threshold-dependent metrics? The results section addresses the first two questions through macro means and category-wise comparisons. The experimental setup documents the calibration procedure used for each model, providing context for the operating-point metrics.

Three broader research directions motivate the next stage of AMEM-Net: transfer to unseen product categories, sensitivity to small or subtle defects, and reliable threshold calibration across product types. These questions require evaluation protocols tailored to cross-category transfer, defect localization, and operating-point selection, respectively.

A. RESEARCH-GAP EVIDENCE MATRIX

Table 2 summarizes the three research directions, their central questions, and suitable evaluation designs.

The first gap concerns a change in the evaluation domain, not simply a larger category count. If an experiment trains separate category-specific memories, then every category in the test set has already contributed normal examples to the learning process. Such a setup measures within-category detection, not transfer to unseen categories. A defensible protocol should specify whether the feature extractor is frozen, whether any normal images from held-out categories can be used for calibration, and whether the comparator methods are evaluated under the same restriction.

The second direction concerns localization. Image-level AUROC and pixel-level localization describe different aspects of detection: a small number of high-scoring patches may rank a defective image correctly without delineating the entire defect. Ground-truth masks and defect-region measurements are useful for analysing sensitivity to small or low-contrast defects.

The third gap concerns the operating point. A threshold chosen using synthetic pseudo-anomalies optimizes separation for that synthetic distribution, while a threshold chosen from normal scores alone controls a different property. The intended application should define which risk matters most. For example, a quality-control line may specify a maximum acceptable false-alarm rate, while another application may prioritize catching nearly every defect. The calibration procedure should be chosen before inspecting test labels and should be applied consistently across compared models.

IV. AMEM-NET METHODOLOGY

A. OVERVIEW

The architecture has two phases. During memory construction, only normal training images are used to extract dense features, cluster global image descriptors, and populate four expert memories. During router training, the memories remain fixed while the router is optimized with normal and synthetic pseudo-anomalous inputs. At inference, the model extracts patch descriptors, predicts expert weights, computes memory distances and feedback weights, and fuses the resulting patch scores. Fig. 1 summarizes this process.

B. FROZEN MULTI-SCALE FEATURE EXTRACTOR

The backbone is an ImageNet-pretrained WideResNet50-2 whose parameters are frozen. The architecture follows residual-network designs, and its pretrained weights originate

TABLE 1. Conceptual Comparison With Representative Industrial Anomaly-Detection Methods

Method	Main representation	Anomaly evidence	Relation to this study
MemAE [12]	Encoder features constrained by a learned memory	Reconstruction discrepancy after memory-guided reconstruction	Provides a memory-augmented reconstruction baseline; AMEM-Net instead scores frozen backbone patch features directly.
PaDiM [5]	Pretrained patch features with location-dependent distribution statistics	Deviation from an estimated normal feature distribution	Represents a distribution-estimation alternative to nearest-memory retrieval.
PatchCore [6]	Representative bank of normal patch descriptors	Nearest-neighbor feature distance	The closest evaluated baseline; both approaches use pretrained features and normal memories, while AMEM-Net adds expert partitioning and adaptive score fusion.
DRAEM [13]	Joint reconstruction and discriminative embedding trained with simulated anomalies	Learned discriminative anomaly evidence and reconstruction-related features	Motivates synthetic perturbations but uses a different architecture and training objective.
FastFlow [16]	Feature maps modeled with 2-D normalizing flows	Feature likelihood under a learned density model	Illustrates density-based feature modeling rather than explicit expert memory matching.
UniAD [21]	Unified model across multiple object categories	Reconstruction-based feature prediction with attention mechanisms	Relevant to multi-category learning; AMEM-Net's current evaluation remains category-specific.
WinCLIP [19]	Vision–language representations and window-level features	Similarity-based zero-/few-shot anomaly evidence	Relevant to broader generalization settings but not a directly matched baseline for the present protocol.
EfficientAD [17]	Lightweight student–teacher feature prediction with an auxiliary autoencoder	Prediction discrepancy and global anomaly evidence	Highlights latency and deployment cost as important dimensions for industrial use.
AMEM-Net (this work)	Frozen multi-scale features with four image-mode expert memories	Router-weighted nearest-memory distance fused with minimum-expert distance	The design under investigation; controlled ablations are still needed to isolate the effect of each component.

TABLE 2. Research Directions and Evaluation Designs

Research direction	Research question	Evaluation design	Relevant measures
Generalization to unseen product categories	Can the detector transfer to categories excluded from training and memory construction?	Hold out complete categories and exclude them from memory construction, router training, and threshold selection.	Image-level AUROC, AUC-PR, category-wise comparisons.
Sensitivity to small or subtle defects	How does detection vary with defect area and contrast?	Group defects using ground-truth masks and predefined defect-size or contrast measures.	Pixel AUROC, pixel AUPR, region overlap.
Threshold reliability	How do calibration choices affect operating points across categories?	Compare predeclared calibration rules using validation data and freeze thresholds before test evaluation.	False-positive rate, recall, precision, F1-score, accuracy.

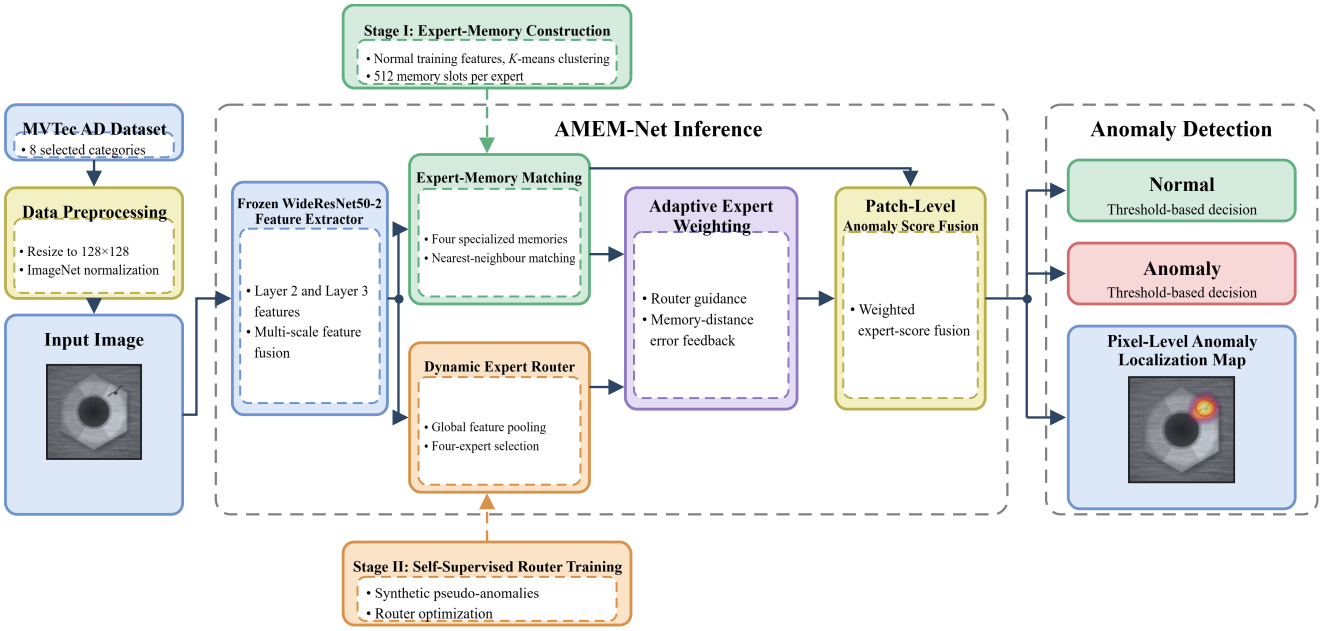


FIGURE 1. AMEM-Net architecture and data flow.

from large-scale visual recognition training [37]–[39]. Given a preprocessed image x, the stem and Layer 1 produce intermediate activations, Layer 2 produces $F^{(2)} \in \mathbb{R}^{512\times h\times w}$, and Layer 3 produces $F^{(3)} \in \mathbb{R}^{1024\times h/2\times w/2}$ under the configured input scale. Layer 3 is bilinearly upsampled to the spatial resolution of Layer 2, and the tensors are concatenated:

$$F(x) = \mathrm{Concat}\left(F^{(2)}, \mathrm{Up}(F^{(3)})\right), \tag{1}$$

where $F(x) \in \mathbb{R}^{1536\times h\times w}$. This design combines a relatively local feature level with a deeper semantic feature level while keeping the backbone fixed. The implementation resizes input images to 128×128 pixels and applies ImageNet normal-

ization. This resolution is part of the reported experimental configuration and should not be confused with the native image resolution of MVTec AD.

For patch index p, the descriptor is normalized to unit length:

$$\mathbf{q}_p = \frac{\mathbf{f}_p}{\|\mathbf{f}_p\|_2 + \epsilon}, \tag{2}$$

where $\mathbf{f}_p \in \mathbb{R}^{1536}$ is the corresponding column of the feature map after spatial flattening. The normalization permits cosine similarity to be used consistently when comparing query descriptors with stored memory vectors.

C. CONSTRUCTION OF FOUR EXPERT MEMORIES

Let $g(x)$ be the normalized global descriptor obtained by spatially averaging the feature map:

$$g(x) = \mathrm{Norm}\left(\frac{1}{hw}\sum_{p=1}^{hw}\mathbf{f}_p\right). \tag{3}$$

MiniBatch K-Means with four clusters, based on standard K-means clustering and its scalable mini-batch variant [40], [41] is fitted to global descriptors from the normal training split. The cluster assignment of an image determines which expert receives that image's patch descriptors. The procedure therefore specializes the memories by image-level normal modes, rather than applying K-Means directly to every patch vector.

For each expert $e \in \{1,2,3,4\}$, the implementation samples 512 normalized patch descriptors from the assigned normal images. If a cluster has fewer than 512 available descriptors, sampling with replacement fills the memory; if it has more, random sampling selects 512. The resulting memory is $\mathcal{M}_e = \{\mathbf{m}_{e,j}\}_{j=1}^{512}$. The total configured capacity is 2048 stored descriptors per category. The memory is constructed before router optimization and stored as a model buffer rather than optimized directly by gradient descent.

This construction makes expert specialization a design assumption that should be tested, not a semantic guarantee. A cluster is defined by global feature similarity, so it need not correspond to a named part, a defect family, or a human-interpretable product mode. Establishing meaningful specialization would require inspecting assignments, comparing within-expert distributions, and evaluating ablations that vary the number of experts and memory capacity.

D. NEAREST-MEMORY DISTANCES

For a normalized query patch $\mathbf{q}_p$ and expert memory $\mathcal{M}_e$, the cosine distance to the closest stored representative is

$$d_{p,e} = 1 - \max_{\mathbf{m}\in\mathcal{M}_e} \mathbf{q}_p^\top \mathbf{m}. \tag{4}$$

The expert-level average distance for an image is

$$\bar{d}_e = \frac{1}{hw}\sum_{p=1}^{hw} d_{p,e}. \tag{5}$$

A low distance indicates that the expert contains a similar normal feature; a high distance indicates that the feature is poorly represented by that memory. The code evaluates experts sequentially to limit temporary memory use. The memory bank itself is fixed during inference.

E. DYNAMIC ROUTER

The router receives $g(x)$ and applies a two-layer multilayer perceptron. Its hidden dimension is 768, followed by GELU activation and layer normalization. The output layer produces four logits, and softmax converts them into expert weights:

$$\mathbf{r}(x) = \mathrm{softmax}\left(W_2\ \mathrm{LN}(\mathrm{GELU}(W_1 g(x))) + b_2\right). \tag{6}$$

Here $\mathbf{r}(x) \in \mathbb{R}^4$, with nonnegative components summing to one. The router is the trainable part of AMEM-Net; the feature extractor and expert memories are frozen. This limits trainable parameters but means that any adaptation occurs through expert weighting rather than learning a new feature representation.

F. MEMORY-DISTANCE FEEDBACK AND SCORE FUSION

The implementation derives a feedback distribution from the mean distance of each expert:

$$\phi_e(x) = \frac{\exp(\bar{d}_e/T)}{\sum_{j=1}^{4}\exp(\bar{d}_j/T)}, \qquad T = 0.10. \tag{7}$$

Larger average distance therefore gives an expert a larger feedback weight. Router and feedback weights are combined as

$$\alpha_e(x) = \mathrm{softmax}_e\left(\log(r_e + \epsilon) + 0.5\log(\phi_e + \epsilon)\right). \tag{8}$$

Equivalently, ignoring the small numerical stabilizer, the combined weight is proportional to $r_e\sqrt{\phi_e}$. The routed anomaly map is

$$A_{\mathrm{route}}(p) = \sum_{e=1}^{4}\alpha_e(x) d_{p,e}, \tag{9}$$

and the most favorable expert distance is

$$A_{\min}(p) = \min_{e\in\{1,2,3,4\}} d_{p,e}. \tag{10}$$

The final patch map uses the implemented fixed fusion weights:

$$A(p) = 0.55 A_{\mathrm{route}}(p) + 0.45 A_{\min}(p). \tag{11}$$

The minimum-distance component is conservative: a patch can receive a low anomaly score if at least one expert represents it well. The routed component retains the image-dependent weighting. The two terms can have different behavior, so their relative coefficients should be tested in a future ablation rather than justified solely by the aggregate results.

The image-level score is the mean of the largest five percent of patch scores:

$$s(x) = \frac{1}{K}\sum_{p\in\mathrm{TopK}(A)} A(p), \qquad K = \max(1, \lfloor 0.05hw \rfloor). \tag{12}$$

This aggregation emphasizes the most anomalous regions while reducing dependence on a single maximum-valued patch. The pixel map is resized or mapped to image coordinates as implemented in the evaluation code. The quantitative comparison in this paper focuses on the five image-level metrics exported for all evaluated models and categories.

G. INFERENCE PROCEDURE

The inference path follows the equations above and uses the full memory capacity for each expert. First, the input image is resized and normalized, then the frozen backbone produces the multi-scale feature map. Each spatial descriptor is normalized and compared with all stored descriptors in each expert memory. The router weights and distance-derived feedback weights are combined to obtain the input-specific expert coefficients. These coefficients are applied to the expert distance maps, which are fused with the minimum-distance map. Finally, the largest five percent of patch scores are averaged to produce the image-level score, while the fused patch map retains spatial information for visualization.

This procedure separates the learned routing decision from the stored normal feature representation. The feature extractor supplies the descriptors, the expert memories provide category-specific normal references, and the router adapts the mixture of their distance maps for each image. The final minimum-distance term retains a route for patches that are well represented by any expert, while the routed term captures the broader weighted response across all experts. The top-patch aggregation then converts the spatial score field into one image-level decision score without requiring a separate RGB decoder.

H. SELF-SUPERVISED ROUTER OBJECTIVE

Synthetic pseudo-anomalies are created from normal training images using three local perturbation families: cut-and-paste with intensity change, a rectangular region with added Gaussian noise, and a local contrast transformation. The corrupted region is selected randomly. The resulting masks are synthetic training constructs and are not real MVTec defect annotations.

For normal features, the code defines a router target distribution based on the negative expert distances:

$$\mathbf{t}(x) = \mathrm{softmax}\left(-\bar{\mathbf{d}}(x)/T\right). \tag{13}$$

The router prediction is encouraged to match this target through a KL-divergence term. A margin term compares the combined expert score for a normal input with the score for its pseudo-anomalous version. Let s_N and s_P denote the routed expert-distance summaries for normal and pseudo-anomalous inputs, respectively. The margin loss is

$$\mathcal{L}_{\mathrm{margin}} = \max(0, m + s_N - s_P), \qquad m = 0.08. \tag{14}$$

A balance penalty discourages the mean router usage from becoming excessively concentrated in a small number of experts. If $\bar{\mathbf{r}}$ denotes mean router usage across a batch and $H(\bar{\mathbf{r}})$ its entropy, the implementation uses

$$\mathcal{L}_{\mathrm{bal}} = \max\left(0, 0.60 \log 4 - H(\bar{\mathbf{r}})\right). \tag{15}$$

The total loss is

$$\mathcal{L} = 0.5\mathcal{L}_{\mathrm{KL}} + 0.5\mathcal{L}_{\mathrm{margin}} + 0.01\mathcal{L}_{\mathrm{bal}}. \tag{16}$$

Only router parameters are updated. The code uses AdamW with learning rate 2×10^{-4}, weight decay 10^{-4}, cosine annealing, gradient clipping at 1.0, and 50 configured epochs. During router training, the distance calculation is limited to 32 memory slots per expert for computational efficiency; inference uses the full 512 slots per expert. This distinction is important for exact reproduction.

I. VALIDATION THRESHOLD AND FINAL DECISION

A continuous anomaly score and a binary decision are different outputs. The AMEM-Net implementation obtains normal scores from held-out normal validation images and generates four pseudo-anomaly variants per validation batch. It selects a threshold by maximizing Youden's statistic over candidate thresholds; the index was originally proposed for rating diagnostic tests [42]:

$$J(\tau) = \mathrm{TPR}_P(\tau) - \mathrm{FPR}_N(\tau), \tag{17}$$

where the positive validation examples are synthetic pseudo-anomalies and the negative examples are normal validation images. The selected threshold is then frozen before the test set is scored. No real test labels are used to choose the AMEM-Net threshold.

This calibration has a limitation: synthetic pseudo-anomalies are only an approximation to real industrial defects. If their texture, scale, or contrast differs from real defects, the chosen threshold may not produce the same operating point on the test distribution. In addition, the other models use a normal-score quantile threshold in the current notebook, and PatchCore uses a 99.5th-percentile normal validation threshold. The resulting threshold-dependent metrics are not a perfectly controlled comparison. Threshold calibration is treated as an implementation protocol, not as an independent novelty claim.

J. ALGORITHMIC SUMMARY AND COMPUTATIONAL CONSIDERATIONS

Algorithm 1 summarizes the inference path implemented in the project. It separates memory construction, router optimization, and test inference so that the normal training data, validation calibration data, and test data have distinct roles.

The memory lookup dominates the distance computation. Let $P = hw$ be the number of query patches, $E = 4$ the number of experts, $M = 512$ the number of memory slots per expert, and $C = 1536$ the feature dimension. A direct similarity calculation has a leading cost proportional to $O(EPMC)$ and stores or streams similarity values proportional to $O(PM)$ for one expert at a time. The implementation processes experts sequentially, which reduces peak temporary memory relative to computing a single dense tensor for all experts. The memory storage itself is EMC floating-point values per category.

Algorithm 1: AMEM-Net inference for one image

Input: Image x; frozen feature extractor f; expert memories $\{\mathcal{M}_e\}_{e=1}^{4}$; trained router $r(\cdot)$.
Output: Image anomaly score $s(x)$ and patch anomaly map A.

1. Resize x to 128×128 and apply ImageNet normalization.
2. Extract Layer-2 and Layer-3 features; upsample Layer 3 and concatenate to obtain $F(x)$.
3. Flatten and normalize the patch descriptors $\{\mathbf{q}_p\}$.
4. Compute the normalized global feature $g(x)$ and router probabilities $\mathbf{r}(x)$.
5. For each expert e, compute $d_{p,e} = 1 - \max_{\mathbf{m}\in\mathcal{M}_e} \mathbf{q}_p^{\top}\mathbf{m}$ for all patches.
6. Compute the mean expert distances $\bar{d}_e$ and feedback weights $\phi_e = \mathrm{softmax}(\bar{d}_e/T)$.
7. Combine router and feedback weights using $\alpha = \mathrm{softmax}(\log(\mathbf{r}+\epsilon) + 0.5\log(\boldsymbol{\phi}+\epsilon))$.
8. Compute $A_{\mathrm{route}}(p) = \sum_e \alpha_e d_{p,e}$ and $A_{\min}(p) = \min_e d_{p,e}$.
9. Fuse the maps: $A(p) = 0.55A_{\mathrm{route}}(p) + 0.45A_{\min}(p)$.
10. Average the largest five percent of patch values to obtain $s(x)$.
11. If a binary decision is required, compare $s(x)$ with the threshold fixed using validation data only.

With the configured dimensions, the four memories contain $4 \times 512 \times 1536$ values; at 32-bit precision, the raw feature storage is approximately 12 MiB, excluding model parameters and runtime buffers.

This estimate is a theoretical storage calculation, not a measured runtime or complete GPU-memory measurement. The notebook also stores intermediate feature maps, model weights, optimizer state, and data batches, which contribute to the actual memory footprint. A valid efficiency comparison should report peak allocated GPU memory, wall-clock training time, median inference latency, and throughput measured under the same hardware and batch-size conditions for all models. Without those measurements, no claim of faster inference or lower overall computational cost is made.

One implementation detail affects the interpretation of router training. The router probabilities are computed from the normal image's global feature in the training batch. The pseudo-anomaly image is separately encoded to obtain its expert distances, but the same normal-image router output is reused when calculating the normal and pseudo-anomaly routed summaries in the margin term. This is the behavior of the current code and should be preserved in any faithful description or changed deliberately in a revised experiment. It should not be silently rewritten as if the router were independently evaluated on the pseudo-anomaly input during the margin comparison.

V. EXPERIMENTAL SETUP

A. DATASET AND CATEGORY SELECTION

The experiment uses the MVTec AD dataset, which includes fifteen object and texture categories with defect-free training images and test images that may contain anomalies [1], [2]. The present notebook selects eight categories: bottle, carpet, hazelnut, metal nut, wood, leather, cable, and transistor. The selected set includes both object-like and texture-like categories, but excludes seven of the fifteen official categories. All reported macro means are therefore averages over the eight selected categories only.

The selected categories follow the dataset directory configuration in the notebook. The paper reports macro means over these eight categories, with each category receiving equal weight.

B. PREPROCESSING AND SPLIT

All images are resized to 128×128 and normalized using the standard ImageNet channel means and standard deviations. For each category, the notebook randomly permutes the normal training images with seed 42 and holds out ten percent for validation. The remaining ninety percent forms the training subset. Training images for the autoencoder baselines receive random horizontal flips, rotations up to five degrees, and mild color jitter; validation and test images use deterministic resizing and normalization. The test set retains the original normal/anomalous labels for final evaluation only.

The fixed seed is used for Python, NumPy, and PyTorch random generators. The notebook also enables cuDNN benchmarking. The input size, batch size of two, 50-epoch schedule, and checkpoint-resume mechanism are configured in the notebook, and the environment details are summarized in Table 3.

C. COMPARED MODELS

Five model configurations are included in the reported results. CNN-AE is a convolutional encoder–decoder trained with reconstruction loss. ResNet-AE uses a WideResNet50-2 encoder, with the stem and Layer 1 frozen and later layers trainable. MemAE uses a learned memory module between its encoder and decoder and is trained with reconstruction loss plus an entropy term. PatchCore uses frozen WideResNet50-2 features, a nearest-neighbor memory, and a coreset ratio configured as 0.25 with up to nine nearest neighbors. AMEM-Net uses the four expert memories and trainable router described in Section IV.

The baseline labels follow the project results table. The ResNet-AE configuration uses a WideResNet50-2 encoder, as detailed below.

D. EVALUATION METRICS

The reported table provides five image-level metrics. AUROC measures ranking quality over thresholds. F1-score is the harmonic mean of precision and recall. Precision is the proportion of predicted anomalous images that are anomalous, recall is the proportion of anomalous images detected, and accuracy is the proportion of all images classified correctly. Precision, recall, F1, and accuracy depend on the chosen threshold; AUROC does not depend on a single threshold.

The main results tables focus on the five image-level metrics exported for all model/category combinations. This consistent presentation allows direct inspection of the image-level detection results across the eight selected categories. Pixel-level localization metrics can complement these measures in a follow-up analysis using anomaly maps and the MVTec ground-truth masks.

E. EVALUATION PROTOCOL, FAIRNESS, AND AGGREGATION

The data flow separates fitting, calibration, and testing. The feature memories are constructed from normal training images only, and the router is optimized using normal images and generated pseudo-anomalies. Normal validation images are reserved for threshold selection. Real test labels are used only after the model and threshold have been fixed. This separation is necessary because threshold selection based on test labels would turn the test set into a tuning set and inflate operating-point metrics. The implementation reserves validation data for threshold selection and uses the fixed threshold for test evaluation.

The compared models use different internal architectures, so the results compare the implemented systems as a whole. The architecture details and calibration procedures are documented to make these distinctions clear. CNN-AE and MemAE learn reconstruction models, the ResNet-AE label refers to an autoencoder whose encoder is based on WideResNet50-2, and PatchCore and AMEM-Net use pretrained feature distances. Their parameter counts, trainable components, and optimization procedures differ. A fair report should describe these differences and avoid attributing every metric gap to one architectural component. Component-wise comparisons provide a complementary way to examine the effect of individual design choices.

The threshold protocol is another source of non-equivalence. AMEM-Net selects its threshold using Youden's J on normal validation scores and synthetic pseudo-anomalies. The baseline implementations use a high quantile of normal validation scores. The two rules optimize different properties: the former attempts to separate two validation score distributions, while the latter sets a cut-off relative to normal scores. The calibration procedure is therefore reported alongside F1, precision, recall, and accuracy; AUROC provides a threshold-independent view of ranking quality.

The reported macro means give each of the eight categories equal weight. This choice prevents categories with more test images from dominating the aggregate, but it does not represent a single pooled production stream. A pooled metric can differ because category sizes and anomaly prevalence differ. Both forms of aggregation can be informative, provided that the manuscript names the aggregation rule and does not mix category means with pooled-image scores. Accordingly, the main aggregate table reports macro means, with each of the eight categories receiving equal weight.

Finally, all models use the configured 128×128 input resolution in the current experiment. Resizing can suppress very small defects and alter the texture statistics that feature memories observe. The resolution is therefore part of the experimental condition, not a neutral implementation detail. A useful sensitivity study would compare a small set of predeclared resolutions, preserve the same data partitions, and report both image-level ranking metrics and verified localization metrics. Any change in resolution should be applied consistently to the baselines and proposed method. These checks would clarify whether the observed performance is stable under a different inspection resolution and whether the memory design remains useful when fine defect details are retained.

TABLE 3. Implementation Settings Recorded in the Notebook

Setting	Recorded value
Input resolution	128×128
Batch size	2
Configured epochs	50
Random seed	42
Feature backbone	ImageNet WideResNet50-2
Feature levels	Layer 2 and upsampled Layer 3
Feature dimension	1536
Number of experts	4
Memory slots per expert	512
Router memory cap during training	32 slots per expert
Router optimizer	AdamW
Router learning rate	2×10^{-4}
Weight decay	10^{-4}
Pseudo-anomaly margin	0.08
Feedback temperature	0.10
Top-patch fraction	5%
PatchCore coreset ratio	0.25
PatchCore neighbor setting	9

F. IMPLEMENTATION CONFIGURATION

Table 3 lists settings that are explicit in the notebook. The table summarizes the principal settings specified in the implementation. The notebook was configured for a conservative batch size suitable for a GPU with limited memory. It also stores checkpoints and category results after completed steps so interrupted runs can resume. Such checkpointing improves operational reliability but does not, by itself, prove that the evaluation results correspond to a particular checkpoint unless the checkpoint and result metadata are retained together.

G. ADDITIONAL IMPLEMENTATION DETAILS FOR REPRODUCTION

The notebook separates normal training data into training and validation subsets by a seeded random permutation. Ten percent of the category's normal training images are reserved for validation, and the remaining images are used for training and memory construction. The validation subset does not receive random training augmentation. This is an important design choice because validation scores are used to select thresholds; applying random augmentation to validation images would change the score distribution and could make the threshold less stable.

For the autoencoder baselines, the training loss is based on smooth L1 reconstruction loss, with an entropy term for the MemAE attention weights. The CNN-AE encoder uses convolutional blocks with channels increasing from 32 to 512,

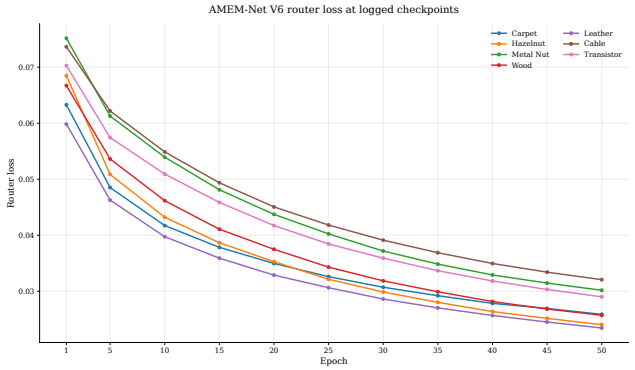


FIGURE 2. AMEM-Net router-training loss from recorded checkpoint values.

and its decoder upsamples the representation to the input resolution. The MemAE implementation adds a 100-item learnable memory with a shrinkage operation to the encoder features before decoding. The ResNet-AE configuration uses a WideResNet50-2 feature extractor, freezes the stem and Layer 1, and decodes from the Layer-4 representation.

The PatchCore implementation registers hooks on intermediate WideResNet50-2 layers, extracts normal patch features, and constructs a reduced memory using the configured coreset ratio. Its nearest-neighbor scorer uses up to nine neighbors. The code calibrates the binary decision threshold from the 99.5th percentile of normal validation image scores. This threshold policy differs from AMEM-Net's Youden-J calibration on normal and pseudo-anomaly scores, so the calibration rule is reported alongside the operating-point metrics.

The AMEM-Net router uses AdamW with cosine learning-rate decay and stores optimizer, scheduler, random-generator state, and loss history in a checkpoint after each completed epoch. The notebook uses PyTorch/torchvision for model execution and scikit-learn for clustering and evaluation utilities [43]–[45]. This checkpointing is designed to resume a long experiment after an interruption. The memory bank is saved separately from the router checkpoint because the memory tensors are larger than the router parameters. The notebook reports PyTorch 2.7.1, torchvision 0.22.1, and an NVIDIA GeForce RTX 3050 laptop GPU with 4.3 GB of reported memory. These values describe the configured execution environment.

Figure 2 plots the router-loss values recorded in the supplied checkpoint history. The curve is presented from the recorded checkpoints rather than interpolated or reconstructed values.

VI. RESULTS

A. CATEGORY-WISE IMAGE-LEVEL PERFORMANCE

The category-wise evaluation reports a value for each combination of eight categories and five model labels. Table 4 reports the recalculated macro means. Each category receives equal weight, so the values are not pooled-image metrics. The AMEM-Net mean AUROC is 97.72%, which is 0.72 percentage points higher than the 97.00% PatchCore mean in the category-wise results. The AMEM-Net mean F1-score

TABLE 4. Macro-Mean Image-Level Results Across the Eight Selected Categories (%)

Model	AUROC	F1-score	Precision	Recall	Accuracy
CNN-AE	89.43	80.65	84.51	77.16	77.25
ResNet-AE	93.44	88.66	91.61	85.91	85.86
MemAE	92.83	85.75	88.31	83.36	82.73
PatchCore	97.00	92.33	92.36	92.56	90.61
AMEM-Net	**97.72**	**94.84**	**96.47**	**93.28**	**93.37**

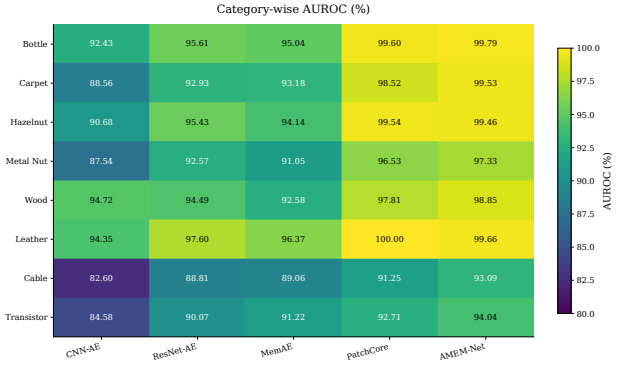


FIGURE 3. Category-wise image-level AUROC (%) across the five evaluated models.

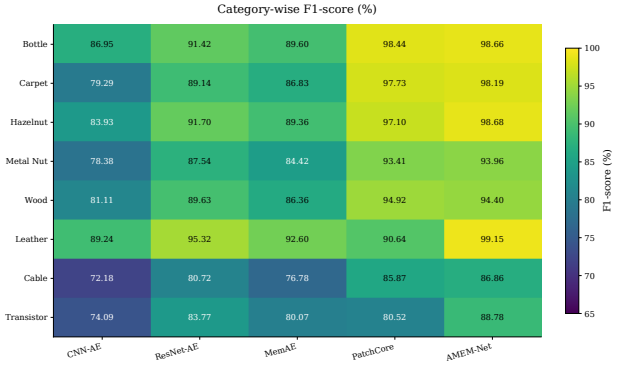


FIGURE 4. Category-wise image-level F1-score (%) across the five evaluated models.

is 94.84%, compared with 92.33% for PatchCore. The mean precision and accuracy are also higher for AMEM-Net in the current evaluation.

AUROC summarizes ranking quality across thresholds, whereas F1-score, precision, recall, and accuracy describe the selected operating point. The 0.72-percentage-point difference in macro-mean AUROC is therefore reported alongside the complete category-wise table and the threshold-dependent metrics.

Figures 3 and 4 show category-level AUROC and F1 patterns. Figure 5 provides the category-wise F1-score comparison across models. Figure 6 presents the five macro-mean metrics, while Fig. 7 reports AUROC separately for each category. These plots support a more nuanced interpretation than the average alone: performance differs by category, and the proposed model is not the highest-scoring method for every category and metric.

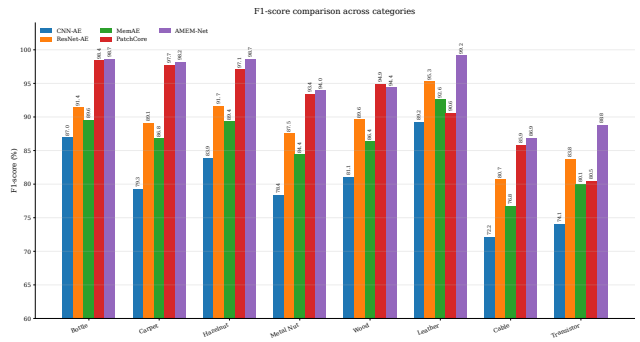


FIGURE 5. Category-wise image-level F1-score comparison across the five evaluated models.

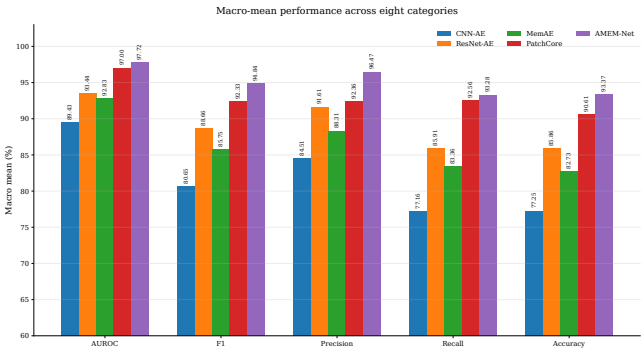


FIGURE 6. Macro-mean image-level performance across the eight evaluated categories.

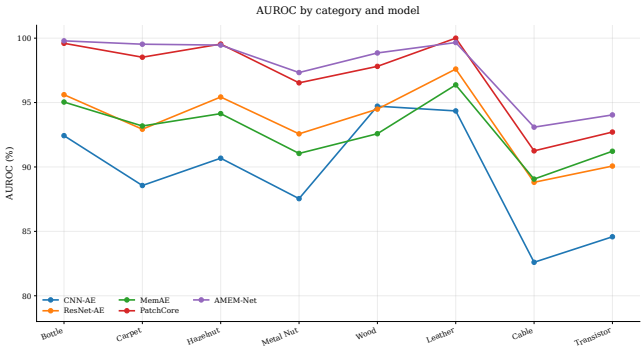


FIGURE 7. Category-wise image-level AUROC for the five evaluated models.

B. MACRO-METRIC DIFFERENCES FROM BASELINES

Table 5 reports the difference between AMEM-Net and each baseline using the macro means across the eight categories. Each value is calculated as the AMEM-Net mean minus the corresponding baseline mean, in percentage points. Positive values indicate a higher numerical value for AMEM-Net on that metric. The table complements the absolute macro means by showing how the size of the difference depends on the selected comparator and metric.

The largest macro-mean differences occur against the reconstruction-based baselines, particularly for recall and accuracy. Relative to CNN-AE, the AMEM-Net macro means are higher by 8.29 percentage points in AUROC and 16.12 points in both recall and accuracy. Relative to ResNet-AE and MemAE, the differences are smaller but remain positive

TABLE 5. AMEM-Net Macro-Mean Differences From Baselines (Percentage Points)

Comparison	AUROC	F1	Precision	Recall	Accuracy
AMEM-Net − CNN-AE	+8.29	+14.19	+11.96	+16.12	+16.12
AMEM-Net − ResNet-AE	+4.28	+6.18	+4.86	+7.37	+7.51
AMEM-Net − MemAE	+4.89	+9.09	+8.16	+9.92	+10.64
AMEM-Net − PatchCore	+0.72	+2.51	+4.11	+0.72	+2.76

across the five reported metrics. These aggregate comparisons describe the values in the project results table under the stated experimental configurations.

PatchCore is the closest comparison because it also uses frozen pretrained patch features and a memory of normal representations. The macro AUROC difference is 0.72 percentage points, while the F1 difference is 2.51 points. Precision differs by 4.11 points, recall by 0.72 points, and accuracy by 2.76 points. The difference pattern is useful because AUROC measures ranking across thresholds, whereas the other four metrics describe decisions at a selected operating point. Reading the metrics together provides a clearer view of the comparison than relying on one number alone.

C. CATEGORY-LEVEL DIFFERENCES

The category-wise AUROC results show AMEM-Net with higher values than PatchCore for bottle, carpet, metal nut, wood, cable, and transistor. PatchCore has higher values for hazelnut and leather. The magnitude of the difference varies by category: the gains on some categories are modest, while cable and transistor show larger differences in the category-wise results. This variation suggests that a single mean does not capture the entire behavior of the method.

The F1-score results show a larger aggregate gap, but F1 depends on the operating threshold. AMEM-Net selects its threshold using normal validation scores and synthetic pseudo-anomalies, while the other models use a high quantile of normal validation scores. The calibration methods can yield different recall–false-alarm trade-offs. Consequently, the F1 difference should not be attributed exclusively to the expert-memory architecture. A controlled experiment should evaluate all methods with a shared threshold-selection protocol, report threshold-free metrics separately, and include per-category false-positive rates at the chosen operating points.

AUROC and the operating-point metrics describe different aspects of detection. AUROC measures the ranking of normal and anomalous samples across thresholds, whereas precision, recall, F1-score, and accuracy are calculated at a selected threshold. Differences between these measures are therefore possible and should be interpreted together with the calibration procedure described in the experimental setup. The paper retains the category-wise values from the project results table and reports them consistently across all five model labels.

D. MACRO-MEAN COMPARISON AND AUROC GAIN

Figure 8 visualizes the difference between AMEM-Net AUROC and the strongest available baseline for each category. Positive differences indicate a higher AMEM-Net AUROC in the reported table; negative differences indicate a higher

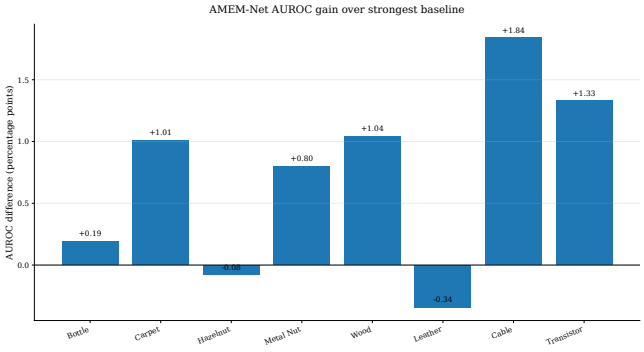


FIGURE 8. Category-wise AUROC difference from the strongest baseline, in percentage points.

TABLE 6. AMEM-Net Validation Calibration Values

Category	Validation pseudo-gap	Threshold
Bottle	0.9237	0.165508
Carpet	0.7411	0.557766
Hazelnut	0.8526	0.454136
Metal nut	0.9432	0.425281
Wood	0.7138	0.492617
Leather	0.8862	0.463827
Cable	0.6325	0.538194
Transistor	0.7743	0.487306

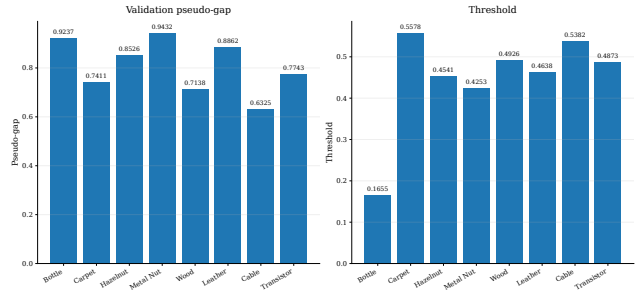


FIGURE 9. Validation calibration values for the eight evaluated categories.

baseline value. The comparison is presented alongside the exact category-wise values. Since the results are rounded to two decimal places, differences are reported at the same precision throughout the paper.

E. THRESHOLD CALIBRATION VALUES

The calibration results contain one validation pseudo-gap value and one threshold for each category. These values are included for traceability in Table 6. They are not test metrics and should not be interpreted as a measure of anomaly-detection accuracy. The threshold depends on the score scale, so thresholds are not directly comparable across categories or models. The pseudo-gap summarizes the separation produced by the synthetic perturbation procedure during validation.

Figure 9 visualizes the category-specific validation calibration values alongside the tabulated values. It is intended to show how the selected thresholds vary by category; it is not a test-performance plot.

TABLE 7. AMEM-Net Minus PatchCore by Category (Percentage-Point Differences)

Category	AMEM AUROC	PatchCore AUROC	Δ_A	Δ_{F1}
Bottle	99.79	99.60	+0.19	+0.22
Carpet	99.53	98.52	+1.01	+0.46
Hazelnut	99.46	99.54	−0.08	+1.58
Metal nut	97.33	96.53	+0.80	+0.55
Wood	98.85	97.81	+1.04	−0.52
Leather	99.66	100.00	−0.34	+8.51
Cable	93.09	91.25	+1.84	+0.99
Transistor	94.04	92.71	+1.33	+8.26

F. RESULTS SUMMARY

The results show that AMEM-Net has the highest macro mean among the five model labels for each of the five reported image-level metrics. Category-wise analysis adds detail to these aggregate values: AMEM-Net records higher AUROC than PatchCore on six categories, while PatchCore records higher AUROC on hazelnut and leather. Reporting both the macro means and category-level results gives a more complete view of model behaviour than either summary alone.

G. CATEGORY-LEVEL ANALYSIS AND OPERATING-POINT INTERPRETATION

The category-level differences provide useful context for the macro averages. Table 7 compares AMEM-Net with PatchCore directly using the values recorded in the result table. The AUROC difference is computed as $\Delta_A = A_{\mathrm{AMEM}} - A_{\mathrm{PatchCore}}$ in percentage points. The F1 difference is calculated in the same way. Positive values favor AMEM-Net numerically; negative values favor PatchCore. These are descriptive differences from one reported run, not estimates of population-level effects.

AMEM-Net has a higher recorded AUROC on six of the eight categories, while PatchCore is higher on hazelnut and leather. The largest AUROC difference in this table is for cable (+1.84 percentage points), followed by transistor (+1.33), wood (+1.04), and carpet (+1.01). Bottle has only a 0.19-point difference. Such variation is expected when categories differ in structure and texture. Category-wise results also motivate future analysis of expert assignments and the relative effects of the routing and fusion components.

The F1 differences are larger than the AUROC differences for some categories, including leather and transistor. This is consistent with the fact that F1 depends on the selected decision threshold, while AUROC summarizes ranking over thresholds. The calibration rules are documented in the experimental setup so that these measures can be interpreted in context.

The contrast between AUROC and threshold-dependent measures is an important methodological result in itself. A model may rank anomalous images well but choose a threshold that yields too many false positives; conversely, a threshold may achieve high accuracy on a particular class distribution without improving the ranking. Therefore, the manuscript should present AUROC and AUC-PR as ranking measures and report precision, recall, F1, and accuracy

only with an explicit calibration protocol. If the intended application prioritizes a low false-alarm rate, the evaluation should also state that target and report recall at the chosen false-alarm level.

VII. DISCUSSION

A. PRACTICAL INTERPRETATION OF OPERATING-POINT METRICS

The five reported metrics serve different purposes in an inspection workflow. AUROC measures how well a model ranks anomalous images above normal images over a range of thresholds. Precision describes how often an image flagged as anomalous is actually defective, while recall describes the fraction of defective images detected. F1-score summarizes the balance between precision and recall, and accuracy measures the proportion of all images classified correctly at the selected threshold. A deployment team should therefore select the metric that matches the inspection objective rather than treating one metric as a complete description of performance.

The category-wise results illustrate these distinctions. For bottle, PatchCore records 100.00% recall, while AMEM-Net records 98.30% recall and 99.02% precision. This comparison reflects different operating points: one setting detects every anomalous test image in the reported category, while the other has a higher reported precision. For carpet, AMEM-Net records higher recall than PatchCore alongside similar precision. For hazelnut, PatchCore has a slightly higher AUROC, while AMEM-Net has higher F1, precision, recall, and accuracy in the reported table. The comparison shows why ranking metrics and threshold-dependent metrics should be considered together.

The metal-nut and wood categories also show that no single operating point summarizes every inspection preference. AMEM-Net records higher AUROC and F1 than PatchCore for metal nut, whereas PatchCore has higher recall for that category. For wood, PatchCore records higher F1 and recall, while AMEM-Net has higher AUROC. In applications where missed defects are costly, recall may receive priority; in applications where false alarms interrupt production, precision and false-positive rate may be more important. The appropriate operating point is therefore an application decision informed by validation data and process requirements.

These examples do not change the reported values; they explain how to read them in context. They also suggest a useful presentation practice for industrial anomaly detection: report threshold-independent ranking metrics separately from threshold-dependent operating-point metrics, document the calibration rule, and provide category-level results in addition to macro averages. This makes the evaluation more informative for both researchers comparing model designs and practitioners selecting an inspection configuration.

B. INTERPRETATION OF THE MULTI-EXPERT DESIGN

The design is motivated by the possibility that several normal modes may be easier to represent with separate memories than with a single undifferentiated set. Clustering global image descriptors provides a low-cost way to assign normal images to experts, while retaining local patch descriptors in each memory. The router then adapts the relative contribution of the experts. This arrangement allows expert contributions to vary with the input. Component-wise comparisons can further quantify the relative effects of clustering, memory size, routing, score fusion, and threshold calibration.

Ablation studies are therefore essential. At minimum, the full system should be compared with (i) a single memory containing the same total number of representatives, (ii) four memories with uniform weights, (iii) four memories with router weights but no feedback, (iv) the routed score without the minimum-distance term, (v) the minimum-distance term alone, and (vi) alternative expert counts. All variants should use identical image splits and a shared threshold protocol. Such tests would show whether the multi-expert design adds value beyond increasing memory capacity or changing score aggregation.

C. METRIC-WISE CATEGORY PATTERNS

The category-wise table shows that AMEM-Net records the highest AUROC in six categories: bottle, carpet, metal nut, wood, cable, and transistor. PatchCore has the highest AUROC for hazelnut and leather. The largest AMEM-Net–PatchCore AUROC difference is 1.84 percentage points for cable, followed by 1.33 points for transistor. The differences for bottle and hazelnut are smaller in magnitude, illustrating that the effect varies across product types rather than remaining constant across the benchmark subset.

For F1-score, AMEM-Net has the highest value in seven categories, with wood as the exception. The precision column follows a similar pattern: AMEM-Net leads on bottle, hazelnut, metal nut, leather, cable, and transistor, while PatchCore has higher precision on carpet and wood. Recall shows a different distribution. PatchCore has higher recall for bottle, metal nut, wood, leather, and cable, whereas AMEM-Net has higher recall for carpet, hazelnut, and transistor. This distribution makes clear why precision, recall, and F1 should be read together rather than interpreted as interchangeable indicators.

Accuracy is highest for AMEM-Net in seven categories, while PatchCore has the highest value for wood. These category-level outcomes align with the macro table, where AMEM-Net has the highest average across the eight categories for all five reported image-level metrics. At the same time, the category-wise view preserves important differences in operating-point behaviour, especially where a model's recall and precision move in opposite directions.

The patterns are consistent with the motivation for input-dependent expert weighting: product categories contain different textures, shapes, and local structures, so a method's relative performance can vary across categories. The result table alone does not assign a causal role to any single component, but it gives a quantitative foundation for further analysis of expert routing and score fusion. In practical

terms, the category-wise metrics also provide a way to select representative examples for a later qualitative review, including categories where AMEM-Net leads in ranking and categories where a baseline retains an advantage.

D. PSEUDO-ANOMALY TRAINING

Synthetic local perturbations allow the router to be trained without using real defective test images. The current implementation uses cut-and-paste, noise, and contrast changes. These perturbations are easy to generate and provide a source of local abnormality during training. However, the synthetic distribution may not match scratches, dents, contamination, structural breaks, or other defects in MVTec AD. A router that separates these synthetic patterns may not learn a reliable boundary for every real anomaly type.

The next experiment should report results separately by defect type and evaluate whether the margin objective generalizes across perturbation families. It should also include a comparison with a router trained without the pseudo-anomaly margin term. A useful sensitivity analysis would vary the margin, feedback temperature, and synthetic patch-size range. These are experiments to conduct, not findings that can be inferred from the existing score table.

E. THRESHOLD SELECTION AND OPERATING CONDITIONS

A threshold converts an anomaly score into a binary decision. In an industrial setting, the threshold often reflects a trade-off between missed defects and false alarms, which can have different costs. A validation-only threshold avoids using test labels to tune the detector, but the calibration data must still represent the intended operating distribution. Synthetic validation anomalies provide one possible calibration source, yet they may be easier or harder than real defects.

AMEM-Net uses Youden's J with normal validation scores and synthetic pseudo-anomalies, whereas the baselines use a high quantile of normal validation scores. The calibration procedure is reported explicitly because it influences the operating point. A shared calibration protocol is a useful complementary analysis; it can set a predeclared false-alarm target using validation data and then report threshold-dependent metrics alongside AUROC and AUC-PR.

F. GENERALIZATION AND DEFECT-SIZE ANALYSIS

The current evaluation is category-specific: normal training images from each selected category are used to build its detector. A complementary cross-category protocol can reserve entire categories for evaluation and exclude them from memory construction, router training, and threshold selection to study transfer to unseen product categories.

Defect-size analysis can further characterize sensitivity to small or low-contrast defects. Ground-truth masks can be used to group defects by area, while pre-defined contrast measures can support a complementary analysis. Pixel-level precision–recall and localization metrics grouped by defect size would provide additional detail beyond image-level scores.

G. REPRODUCIBILITY AND COMPUTATIONAL COST

The notebook specifies a fixed seed, input size, training schedule, memory capacity, optimizer, and principal loss weights. It also includes checkpoint-resume logic and writes result rows as each category-model evaluation completes. These implementation choices support a reproducible workflow for experiments conducted on limited hardware. The principal settings are summarized in Table 3.

A reproducibility package can include a requirements file or environment export, dataset version, split indices, category inventory, model checkpoints or reconstruction scripts, raw test scores, thresholds, prediction masks, metric scripts, and a README describing execution order. Runtime and memory measurements can be reported alongside accuracy metrics when computational efficiency is a study objective.

H. EVALUATION SCOPE

The evaluation is defined around eight MVTec AD categories, a 128×128 input resolution, and five image-level metrics. These settings define the scope of the reported comparison. Results may vary with category selection, image resolution, threshold calibration, and the distribution of products encountered in a deployment setting. Reporting the selected categories, preprocessing, feature backbone, memory settings, and calibration rules makes the experimental conditions explicit. Broader studies can extend the same evaluation to additional categories, localization metrics, alternative resolutions, and data from other industrial environments.

I. COMPONENT-WISE ANALYSIS

The architecture contains several interacting elements: four expert memories, input-dependent routing, memory-distance feedback, and score fusion. These elements are designed to provide complementary information when scoring a test image. A component-wise analysis can compare alternative memory counts, uniform and routed expert weights, feedback-enabled and feedback-free routing, and different score-fusion rules. For a controlled comparison, each variant should retain the same image split, feature extractor, and metric definitions. This analysis offers a direct route to understanding how the architectural choices influence category-wise detection behaviour.

J. STATISTICAL RELIABILITY AND UNCERTAINTY REPORTING

The macro-mean table gives each of the eight categories equal weight, and the category-wise table preserves variation between products and textures. Reporting both views distinguishes average performance from category-specific behaviour. Threshold-dependent measures are interpreted alongside each model's calibration rule, while AUROC provides a threshold-independent ranking measure.

VIII. LIMITATIONS AND FUTURE WORK

Future work can extend the evaluation in three directions. First, a shared threshold-calibration protocol across all models would provide an additional matched operating-point comparison. Second, category-wise localization analysis can pair spatial anomaly maps with ground-truth masks. Third, controlled variations of memory count, router feedback, and score fusion can quantify how each design choice affects performance. These extensions build on the current eight-category image-level results.

A second direction is pixel-level localization analysis. Spatial maps derived from the AMEM-Net feature-distance output can be compared with ground-truth masks using pixel AUROC, pixel AUPR, and region-overlap measures. Presenting the original image, predicted anomaly map, and ground-truth mask together would make localization behaviour easier to interpret.

A third direction is a component-wise study of the multi-expert design. Useful variants include one memory versus four memories, fixed versus dynamic routing, routing with versus without distance feedback, and alternative score-fusion rules. Keeping data splits, feature extraction, and evaluation metrics consistent across variants would clarify the contribution of each design choice. Repeated runs can further characterize the stability of the category-wise results.

Further experiments can address three broader questions: transfer to held-out product categories, sensitivity to small or low-contrast defects, and threshold reliability across product types. Each question calls for a protocol designed specifically for that purpose, for example, category-level holdout, defect-size grouping using masks, or a predefined false-alarm target. Such studies would extend the current within-category evaluation.

Finally, broader comparison should include recent methods appropriate to the task and a second industrial dataset if resources permit. Published numbers should not be placed in the same ranking table as the current results unless the evaluation settings are comparable. The input resolution, category coverage, backbone, training data, and metric definitions can all affect reported performance. A careful comparison should state these differences rather than treating every published score as directly interchangeable.

A. OPERATIONAL CONSIDERATIONS FOR INDUSTRIAL INSPECTION

Although the current evaluation is benchmark-based, the architecture is intended for a setting in which an inspection system must make repeatable decisions over a stream of product images. Several deployment questions remain separate from image-level AUROC. A detector must tolerate normal changes in illumination and pose, keep its false-alarm rate within an acceptable range, produce maps that operators can interpret, and allow the stored normal representation to be updated when the manufacturing process changes. These properties define practical conditions for subsequent evaluation of AMEM-Net.

Calibration drift. The threshold in the current implementation is chosen from held-out normal images and synthetic pseudo-anomalies. If the camera, illumination, material batch, or product alignment changes, the score distribution can shift even when the products remain acceptable. A deployment study should therefore monitor the distribution of scores on verified normal samples and define when recalibration is allowed. Recalibration must be performed using a separate validation stream or an approved normal-only calibration set; operators should not tune the threshold against the final evaluation labels. A threshold version should be stored alongside the model checkpoint and memory bank so that an observed decision can be traced to the exact configuration.

Memory updates. The expert memories represent the normal training distribution at the time of model construction. If a legitimate production change introduces a new normal appearance, the memory may treat it as unusual. A naïve update that adds every low-scoring or high-scoring sample automatically can contaminate the memory with defective examples. Any update strategy should require quality-controlled normal data, retain a versioned copy of the previous memory, and compare performance before and after the update. Dynamic routing selects expert contributions for each input, whereas an online memory-update mechanism would change stored representations during operation; these are distinct design choices.

Localization and operator review. A heatmap can help an operator understand which region contributes to an anomaly score, but a visually bright region is not automatically a correct defect segmentation. A production evaluation should measure overlap with verified defect masks and examine false-positive maps on normal products. Maps should be shown at the original image resolution and with a consistent color scale. When the image score and the localization map disagree, for example, a high image score with diffuse low-intensity regions, the system should have a documented review path rather than silently making a final decision based on the heatmap alone.

Latency and resource constraints. The four expert memories increase the number of nearest-memory comparisons relative to a single memory with a comparable feature dimension. Sequential expert processing reduces temporary memory demand, but it does not remove the extra distance calculations. A practical evaluation should report end-to-end latency including image preprocessing, feature extraction, memory lookup, score aggregation, and thresholding. It should also report throughput at the intended batch size, peak memory, and the effect of memory capacity on detection quality. End-to-end latency, throughput, and peak memory are useful deployment measures to report alongside detection quality.

Failure handling. Industrial inspection systems should record the model version, score, threshold, category, and final decision for each inspected image. They should also define what happens if the image is corrupted, the product category is uncertain, or the model cannot produce a valid score. A

TABLE 8. Category-Wise Results: Bottle, Carpet, Hazelnut, and Metal Nut (%)

Category	Model	AUROC	F1	Precision	Recall	Accuracy
Bottle	CNN-AE	92.43	86.95	89.45	84.59	83.54
Bottle	ResNet-AE	95.61	91.42	93.33	89.59	89.91
Bottle	MemAE	95.04	89.60	91.23	88.03	87.96
Bottle	PatchCore	99.60	98.44	96.92	100.00	97.59
Bottle	AMEM-Net	99.79	98.66	99.02	98.30	98.24
Carpet	CNN-AE	88.56	79.29	82.23	76.56	76.63
Carpet	ResNet-AE	92.93	89.14	91.23	87.14	86.60
Carpet	MemAE	93.18	86.83	89.16	84.62	84.14
Carpet	PatchCore	98.52	97.73	98.85	96.63	96.58
Carpet	AMEM-Net	99.53	98.19	98.75	97.63	97.42
Hazelnut	CNN-AE	90.68	83.93	86.92	81.14	80.01
Hazelnut	ResNet-AE	95.43	91.70	93.89	89.61	89.24
Hazelnut	MemAE	94.14	89.36	91.39	87.41	86.82
Hazelnut	PatchCore	99.54	97.10	98.53	95.71	96.36
Hazelnut	AMEM-Net	99.46	98.68	99.40	97.97	97.87
Metal nut	CNN-AE	87.54	78.38	83.06	74.20	75.04
Metal nut	ResNet-AE	92.57	87.54	90.93	84.40	84.21
Metal nut	MemAE	91.05	84.42	87.24	81.78	80.86
Metal nut	PatchCore	96.53	93.41	95.51	91.40	89.57
Metal nut	AMEM-Net	97.33	93.96	96.85	91.24	91.39

TABLE 9. Category-Wise Results: Wood, Leather, Cable, and Transistor (%)

Category	Model	AUROC	F1	Precision	Recall	Accuracy
Wood	CNN-AE	94.72	81.11	85.18	77.41	78.28
Wood	ResNet-AE	94.49	89.63	93.21	86.31	86.76
Wood	MemAE	92.58	86.36	89.34	83.57	83.24
Wood	PatchCore	97.81	94.92	96.55	93.33	92.41
Wood	AMEM-Net	98.85	94.40	96.40	92.49	92.35
Leather	CNN-AE	94.35	89.24	92.29	86.39	86.29
Leather	ResNet-AE	97.60	95.32	97.22	93.49	93.41
Leather	MemAE	96.37	92.60	94.01	91.23	90.07
Leather	PatchCore	100.00	90.64	82.88	100.00	84.68
Leather	AMEM-Net	99.66	99.15	99.64	98.66	98.96
Cable	CNN-AE	82.60	72.18	78.43	66.85	68.08
Cable	ResNet-AE	88.81	80.72	85.35	76.56	77.07
Cable	MemAE	89.06	76.78	80.78	73.15	72.74
Cable	PatchCore	91.25	85.87	85.87	85.87	82.67
Cable	AMEM-Net	93.09	86.86	89.89	84.03	84.39
Transistor	CNN-AE	84.58	74.09	78.51	70.15	70.11
Transistor	ResNet-AE	90.07	83.77	87.74	80.15	79.66
Transistor	MemAE	91.22	80.07	83.31	77.08	75.99
Transistor	PatchCore	92.71	80.52	83.78	77.50	85.00
Transistor	AMEM-Net	94.04	88.78	91.80	85.95	86.35

conservative deployment may route uncertain cases to human review rather than forcing a binary decision. These controls are outside the current notebook, but they are necessary to translate a benchmark detector into a reliable inspection component.

IX. CONCLUSION

In summary, AMEM-Net combines frozen multi-scale features, four normal patch memories, input-dependent routing, memory-distance feedback, and top-patch score aggregation. Its eight-category image-level results provide a quantitative comparison with four baseline implementations. The analysis also identifies clear directions for extending the method, including matched threshold calibration, pixel-level localization, component-wise evaluation, and transfer to held-out categories. These extensions can build on the implementation and results presented in this paper.

.

APPENDIX A COMPLETE CATEGORY-WISE IMAGE-LEVEL RESULTS

Tables 8 and 9 present the five image-level metrics for all eight categories and five model labels. The values are rounded to two decimal places, and the macro means in the main results section are calculated by giving each category equal weight.

APPENDIX B METRIC DEFINITIONS AND AGGREGATION

For image-level evaluation, each test image receives a continuous score s_i and a binary label $y_i \in \{0, 1\}$. AUROC is the area under the receiver operating characteristic curve formed by varying the score threshold; the ROC framework and its interpretation are discussed by Fawcett [46]. AUC-PR summarizes precision as a function of recall and is often informative when anomalous images are less common than normal images; precision–recall and ROC curves can give different views under class imbalance [47]–[49]. These ranking metrics should be computed from the continuous scores, not from binary predictions produced at a single threshold.

At threshold τ, the predicted label is $\hat{y}_i = \mathbb{1}[s_i \geq \tau]$. With true positives TP, false positives FP, true negatives TN, and false negatives FN, the reported operating-point metrics are

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{18}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{19}$$

$$F_1 = 2\frac{\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}}, \tag{20}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}, \tag{21}$$

$$\text{FPR} = \frac{FP}{FP + TN}. \tag{22}$$

When a denominator is zero, the metric implementation must state how the case is handled. The notebook uses zero-division handling for precision and F1. For macro-mean reporting, category-specific metric values are averaged with equal category weight:

$$\bar{m}_{\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} m_c, \qquad C = 8. \tag{23}$$

This is different from pooling every test image across categories and calculating a single metric on the combined predictions. The table in this paper uses the macro mean over the eight category rows for each model.

For localization analysis, a pixel-level anomaly map $A_i(u, v)$ can be compared with the ground-truth mask $G_i(u, v)$. Pixel AUROC and pixel AUPR treat pixels as score-label pairs, while AUPRO evaluates region overlap across thresholds under a specified false-positive-rate limit. These metrics complement image-level measures by evaluating the spatial correspondence between anomaly scores and annotated defect regions.

ACKNOWLEDGMENT

The authors gratefully acknowledge Vellore Institute of Technology, Chennai, for institutional support and Dr. Reena Roy for research guidance and supervision.

REFERENCES

[1] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD: A comprehensive real-world dataset for unsupervised anomaly detection," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2019, pp. 9592–9600.

[2] MVTec Software GmbH, "MVTec AD: Industrial anomaly detection benchmark dataset." [Online]. Available: https://www.mvtec.com/research-teaching/datasets/mvtec-ad. Accessed: Oct. 15, 2026.

[3] J. Liu, G. Xie, J. Wang, S. Li, C. Wang, F. Zheng, and Y. Jin, "Deep industrial image anomaly detection: A survey," Mach. Intell. Res., vol. 21, pp. 104–135, 2024, doi: 10.1007/s11633-023-1459-z.

[4] Y. Cao, X. Xu, J. Zhang, Y. Cheng, X. Huang, G. Pang, and W. Shen, "A survey on visual anomaly detection: Challenge, approach, and prospect," arXiv:2401.16402, 2024.

[5] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in Proc. Int. Conf. Pattern Recognit. (ICPR), 2021, pp. 475–489.

[6] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 14318–14328, doi: 10.1109/CVPR52688.2022.01392.

[7] G. Wang, S. Han, E. Ding, and D. Huang, "Student-teacher feature pyramid matching for anomaly detection," in Proc. Brit. Mach. Vis. Conf. (BMVC), 2021.

[8] H. Deng and X. Li, "Anomaly detection via reverse distillation from one-class embedding," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 9737–9746.

[9] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in Proc. 37th Int. Conf. Mach. Learn. (ICML), 2020, pp. 1597–1607.

[10] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 9729–9738.

[11] Z. Liu, Y. Zhou, Y. Xu, and Z. Wang, "SimpleNet: A simple network for image anomaly detection and localization," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 20402–20411, doi: 10.1109/CVPR52729.2023.01954.

[12] D. Gong, L. Liu, V. Le, B. Saha, M. R. Mansour, S. Venkatesh, and A. van den Hengel, "Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 1705–1714, doi: 10.1109/ICCV.2019.00179.

[13] V. Zavrtanik, M. Kristan, and D. Skočaj, "DRAEM: A discriminatively trained reconstruction embedding for surface anomaly detection," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021, pp. 8330–8339, doi: 10.1109/ICCV48922.2021.00822.

[14] C.-L. Li, K. Sohn, J. Yoon, and T. Pfister, "CutPaste: Self-supervised learning for anomaly detection and localization," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2021, pp. 9664–9674, doi: 10.1109/CVPR46437.2021.00954.

[15] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "Uninformed students: Student-teacher anomaly detection with discriminative latent embeddings," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 4183–4192, doi: 10.1109/CVPR42600.2020.00424.

[16] J. Yu, Y. Zheng, X. Wang, W. Li, Y. Wu, R. Zhao, and L. Wu, "FastFlow: Unsupervised anomaly detection and localization via 2D normalizing flows," in Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV), 2022, pp. 3943–3952.

[17] K. Batzner, L. Heckler, and R. König, "EfficientAD: Accurate visual anomaly detection at millisecond-level latencies," in Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV), 2024, pp. 128–138, doi: 10.1109/WACV57701.2024.00020.

[18] D. Gudovskiy, S. Ishizaka, and K. Kozuka, "CFLOW-AD: Real-time unsupervised anomaly detection with localization via conditional normalizing flows," in Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV), 2022, pp. 98–107.

[19] J. Jeong, Y. Zou, T. Kim, D. Zhang, A. Ravichandran, and O. Dabeer, "WinCLIP: Zero-/few-shot anomaly classification and segmentation," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 19606–19616, doi: 10.1109/CVPR52729.2023.01878.

[20] Q. Zhou, G. Pang, Y. Tian, S. He, and J. Chen, "AnomalyCLIP: Object-agnostic prompt learning for zero-shot anomaly detection," in Proc. Int. Conf. Learn. Represent. (ICLR), 2024.

[21] Z. You, L. Cui, Y. Shen, K. Yang, X. Lu, Y. Zheng, and X. Le, "A unified model for multi-class anomaly detection," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 35, 2022.

[22] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "Beyond dents and scratches: Logical constraints in unsupervised anomaly detection and localization," Int. J. Comput. Vis., vol. 130, pp. 947–969, 2022, doi: 10.1007/s11263-022-01578-9.

[23] P. Carvalho, M. Lafou, A. Durupt, A. Leblanc, and Y. Grandvalet, "Detecting visual anomalies in an industrial environment: Unsupervised methods put to the test on the AutoVI dataset," Computers in Industry, vol. 163, Art. no. 104151, 2024, doi: 10.1016/j.compind.2024.104151.

[24] L. Ruff, R. A. Vandermeulen, N. Görnitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, "Deep one-class classification," in Proc. 35th Int. Conf. Mach. Learn. (ICML), 2018, pp. 4393–4402.

[25] T. Schlegl, P. Seeböck, S. M. Waldstein, U. Schmidt-Erfurth, and G. Langs, "Unsupervised anomaly detection with generative adversarial networks to guide marker discovery," in Proc. Int. Conf. Inf. Process. Med. Imaging (IPMI), 2017, pp. 146–157, doi: https://doi.org/10.1007/978-3-319-59050-9_12.

[26] S. Akcay, A. Atapour-Abarghouei, and T. P. Breckon, "GANomaly: Semi-supervised anomaly detection via adversarial training," in Proc. Asian Conf. Comput. Vis. (ACCV), 2018, pp. 622–637, doi: https://doi.org/10.1007/978-3-030-20893-6_39.

[27] N. Cohen and Y. Hoshen, "Sub-image anomaly detection with deep pyramid correspondences," arXiv:2005.02357, 2020.

[28] J. Bae, J.-H. Lee, and S. Kim, "PNI: Industrial anomaly detection using position and neighborhood information," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2023, pp. 6373–6383.

[29] Z. Jiang, Y. Zhang, Y. Wang, J. Li, and X. Gao, "FR-PatchCore: An industrial anomaly detection method for improving generalization," Sensors, vol. 24, no. 5, Art. no. 1368, 2024, doi: 10.3390/s24051368.

[30] X. Liu, J. Wang, B. Leng, and S. Zhang, "Dual-modeling decouple distillation for unsupervised anomaly detection," in Proc. ACM Int. Conf. Multimedia (ACM MM), 2024, pp. 5035–5044, doi: 10.1145/3664647.3681669.

[31] X. Li, Z. Zhang, X. Tan, C. Chen, Y. Qu, Y. Xie, and L. Ma, "PromptAD: Learning prompts with only normal samples for few-shot anomaly detection," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2024, pp. 16838–16848.

[32] Y. Wang, J. Peng, J. Zhang, R. Yi, Y. Wang, and C. Wang, "Multimodal industrial anomaly detection via hybrid fusion," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 8032–8041.

[33] A. Costanzino, P. Zama Ramirez, G. Lisanti, and L. Di Stefano, "Multimodal industrial anomaly detection by crossmodal feature mapping," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2024, pp. 17234–17243.

[34] A. Baitieva, D. Hurych, V. Besnier, and O. Bernard, "Supervised anomaly detection for complex industrial images," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2024, pp. 17754–17762.

[35] Y. Zou, J. Jeong, L. Pemula, D. Zhang, and O. Dabeer, "SPot-the-difference self-supervised pre-training for anomaly detection and segmentation," in Proc. Eur. Conf. Comput. Vis. (ECCV), 2022, pp. 392–408, doi: https://doi.org/10.1007/978-3-031-20056-4_23.

[36] P. Bergmann, X. Jin, D. Sattlegger, and C. Steger, "The MVTec 3D-AD dataset for unsupervised 3D anomaly detection and localization," in Proc. 17th Int. Joint Conf. Comput. Vis., Imaging Comput. Graph. Theory Appl. (VISIGRAPP), 2022, pp. 202–213, doi: 10.5220/0010865000003124.

[37] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[38] S. Zagoruyko and N. Komodakis, "Wide residual networks," in Proc. Brit. Mach. Vis. Conf. (BMVC), 2016, pp. 87.1–87.12, doi: 10.5244/C.30.87.

[39] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," Int. J. Comput. Vis., vol. 115, pp. 211–252, 2015, doi: 10.1007/s11263-015-0816-y.

[40] S. P. Lloyd, "Least squares quantization in PCM," IEEE Trans. Inf. Theory, vol. 28, no. 2, pp. 129–137, 1982, doi: 10.1109/TIT.1982.1056489.

[41] D. Sculley, "Web-scale k-means clustering," in Proc. 19th Int. Conf. World Wide Web (WWW), 2010, pp. 1177–1178, doi: 10.1145/1772690.1772862.

[42] W. J. Youden, "Index for rating diagnostic tests," Cancer, vol. 3, no. 1, pp. 32–35, 1950,
[43] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 32, 2019.
[44] PyTorch Contributors, "Torchvision documentation." [Online]. Available: https://pytorch.org/vision/stable/. Accessed: Oct. 15, 2026.
[45] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
[46] T. Fawcett, "An introduction to ROC analysis," Pattern Recognit. Lett., vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.
[47] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in Proc. 23rd Int. Conf. Mach. Learn. (ICML), 2006, pp. 233–240, doi: 10.1145/1143844.1143874.
[48] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.
[49] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.